Direct Methods for Sparse Linear Systems

Fundamentals of Algorithms

Editor-in-Chief: Nicholas J. Higham, University of Manchester

The SIAM series on Fundamentals of Algorithms publishes monographs on state-of-the-art numerical methods to provide the reader with sufficient knowledge to choose the appropriate method for a given application and to aid the reader in understanding the limitations of each method. The monographs focus on numerical methods and algorithms to solve specific classes of problems and are written for researchers, practitioners, and students.

The goal of the series is to produce a collection of short books, written by experts on numerical methods, that include an explanation of each method and a summary of theoretical background. What distinguishes a book in this series is both its emphasis on explaining how to best choose a method, algorithm, or software package to solve a specific type of problem and its descriptions of when a given algorithm or method succeeds or fails.

Editorial Board

Peter Benner
Technische Universität Chemnitz

John R. Gilbert
University of California, Santa Barbara

Michael T. Heath
University of Illinois—Urbana-Champaign

C. T. Kelley
North Carolina State University

Cleve Moler
The MathWorks, Inc.

James G. Nagy
Emory University

Dianne P. O'Leary
University of Maryland

Robert D. Russell
Simon Fraser University

Robert D. Skeel
Purdue University

Danny Sorensen
Rice University

Henry Wolkowicz
University of Waterloo

Series Volumes

Chan, R. H.-F. and Jin, X.-Q., *An Introduction to Iterative Toeplitz Solvers*
Eldén, L., *Matrix Methods in Data Mining and Pattern Recognition*
Hansen, P. C., Nagy, J. G., and O'Leary, D. P., *Deblurring Images: Matrices, Spectra, and Filtering*
Davis, T. A., *Direct Methods for Sparse Linear Systems*
Kelley, C. T., *Solving Nonlinear Equations with Newton's Method*

Timothy A. Davis
University of Florida
Gainesville, Florida

Direct Methods for Sparse Linear Systems

siam
Society for Industrial and Applied Mathematics
Philadelphia

Copyright © 2006 by Society for Industrial and Applied Mathematics.

10 9 8 7 6 5 4 3 2

All rights reserved. Printed in the United States of America. No part of this book may be reproduced, stored, or transmitted in any manner without the written permission of the publisher. For information, write to the Society for Industrial and Applied Mathematics, 3600 Market Street, 6th floor, Philadelphia, PA 19104-2688 USA.

Trademarked names may be used in this book without the inclusion of a trademark symbol. These names are used in an editorial context only; no infringement of trademark is intended.

Maple is a registered trademark of Waterloo Maple, Inc.

MATLAB is a registered trademark of The MathWorks, Inc. For MATLAB product information, please contact The MathWorks, Inc., 3 Apple Hill Drive, Natick, MA 01760-2098 USA, 508-647-7000, Fax: 508-647-7101, i*nfo@mathworks.com, www.mathworks.com*

No warranties, expressed or implied, are made by the publisher, author, and their employers that the programs contained in this volume are free of error. They should not be relied on as the sole basis to solve a problem whose incorrect solution could result in injury to person or property. If the programs are employed in such a manner, it is at the user's own risk and the publisher, author, and their employers disclaim all liability for such misuse.

The algorithms presented in this book were developed with support from various sources, including Sandia National Laboratory, the National Science Foundation (ASC-9111263, DMS-9223088, DMS-9504974, DMS-9803599, and CCR-0203720), The MathWorks, Inc., and the University of Florida.

Library of Congress Cataloging-in-Publication Data

Davis, Timothy A.
Direct methods for sparse linear systems / Timothy A. Davis.
p. cm. — (Fundamentals of algorithms)
Includes bibliographical references and index.
ISBN 978-0-898716-13-9 (pbk.)
1. Sparse matrices. 2. Linear systems. I. Title.

QA188.D386 2006
512.9'434—dc22

2006044387

siam is a registered trademark.

For Connie, Emily, and Timothy J. ("TJ")

Contents

Preface **xi**

1 Introduction **1**
1.1 Linear algebra . . . 2
1.2 Graph theory, algorithms, and data structures . . . 4
1.3 Further reading . . . 6

2 Basic algorithms **7**
2.1 Sparse matrix data structures . . . 7
2.2 Matrix-vector multiplication . . . 9
2.3 Utilities . . . 10
2.4 Triplet form . . . 12
2.5 Transpose . . . 14
2.6 Summing up duplicate entries . . . 15
2.7 Removing entries from a matrix . . . 16
2.8 Matrix multiplication . . . 17
2.9 Matrix addition . . . 19
2.10 Vector permutation . . . 20
2.11 Matrix permutation . . . 21
2.12 Matrix norm . . . 22
2.13 Reading a matrix from a file . . . 23
2.14 Printing a matrix . . . 23
2.15 Sparse matrix collections . . . 24
2.16 Further reading . . . 24
Exercises . . . 24

3 Solving triangular systems **27**
3.1 A dense right-hand side . . . 27
3.2 A sparse right-hand side . . . 29
3.3 Further reading . . . 35
Exercises . . . 35

4 Cholesky factorization **37**
4.1 Elimination tree . . . 38

4.2 Sparse triangular solve . . . 43
4.3 Postordering a tree . . . 44
4.4 Row counts . . . 46
4.5 Column counts . . . 52
4.6 Symbolic analysis . . . 56
4.7 Up-looking Cholesky . . . 58
4.8 Left-looking and supernodal Cholesky . . . 60
4.9 Right-looking and multifrontal Cholesky . . . 62
4.10 Modifying a Cholesky factorization . . . 63
4.11 Further reading . . . 66
Exercises . . . 67

5 Orthogonal methods 69
5.1 Householder reflections . . . 69
5.2 Left- and right-looking QR factorization . . . 70
5.3 Householder-based sparse QR factorization . . . 71
5.4 Givens rotations . . . 79
5.5 Row-merge sparse QR factorization . . . 79
5.6 Further reading . . . 81
Exercises . . . 82

6 LU factorization 83
6.1 Upper bound on fill-in . . . 83
6.2 Left-looking LU . . . 85
6.3 Right-looking and multifrontal LU . . . 88
6.4 Further reading . . . 94
Exercises . . . 95

7 Fill-reducing orderings 99
7.1 Minimum degree ordering . . . 99
7.2 Maximum matching . . . 112
7.3 Block triangular form . . . 118
7.4 Dulmage–Mendelsohn decomposition . . . 122
7.5 Bandwidth and profile reduction . . . 127
7.6 Nested dissection . . . 128
7.7 Further reading . . . 130
Exercises . . . 133

8 Solving sparse linear systems 135
8.1 Using a Cholesky factorization . . . 135
8.2 Using a QR factorization . . . 136
8.3 Using an LU factorization . . . 138
8.4 Using a Dulmage–Mendelsohn decomposition . . . 138
8.5 MATLAB sparse backslash . . . 140
8.6 Software for solving sparse linear systems . . . 141
Exercises . . . 144

9 CSparse **145**
9.1 Primary CSparse routines and definitions 146
9.2 Secondary CSparse routines and definitions 149
9.3 Tertiary CSparse routines and definitions 154
9.4 Examples 158

10 Sparse matrices in MATLAB **169**
10.1 Creating sparse matrices 169
10.2 Sparse matrix functions and operators 172
10.3 CSparse MATLAB interface 176
10.4 Examples 182
10.5 Further reading 186
Exercises 186

A Basics of the C programming language **187**

Bibliography **195**

Index **211**

Preface

This book presents the fundamentals of sparse matrix algorithms, from theory to algorithms and data structures to working code. The focus is on direct methods for solving systems of linear equations; iterative methods and solvers for eigenvalue problems are beyond the scope of this book.

The goal is to impart a working knowledge of the underlying theory and practice of sparse matrix algorithms, so that you will have the foundation to understand more complex (but faster) algorithms. Methods that operate on dense submatrices of a larger sparse matrix (multifrontal and supernodal methods) are much faster, but a complete sparse matrix package based on these methods can be tens of thousands of lines long. The sparse LU, Cholesky, and QR factorization codes in MATLAB®, for example, total about 100,000 lines of code. Trying to understand the sparse matrix technique by starting with such huge codes is a daunting task. To overcome this obstacle, a sparse matrix package, CSparse,[1] has been written specifically for this book.[2] It can solve $Ax = b$ when A is unsymmetric, symmetric positive definite, or rectangular, using about 2,200 lines of code. Although simple and concise, it is based on recently developed methods and theory. All of CSparse is printed in this book. Take your time to read and understand these codes; do not gloss over them. You will find them much easier to comprehend and learn from than their larger (yet faster) cousins. The larger packages you may use in practice are based on much of the theory and some of the algorithms presented more concisely and simply in CSparse. For example, the MATLAB statement `x=A\b` relies on the theory and algorithms from almost every section of this book. Parallel sparse matrix algorithms are excluded, yet they too rely on the theory discussed here.

For the computational scientist with a problem to solve using sparse matrix methods, these larger packages may be faster, but you need to understand how they work to use them effectively. They might not have every function needed to interface them into your application. You may need to write some code of your own to manipulate your matrix prior to or after using a large sparse matrix package. One of the goals of this book is to equip you for these tasks. The same question applies to MATLAB. You might ask, "*What is the most efficient way of solving my sparse matrix problem in MATLAB?*" The short answer is to always operate on whole matrices, large submatrices, or column vectors in MATLAB and to not rely

[1] CSparse: a Concise Sparse matrix package.

[2] The index gives page numbers in bold that contain CSparse and related software.

heavily on accessing the rows or individual entries of a sparse matrix. The long answer to this question is to read this book. MATLAB and the C programming language are a strong emphasis of this book. In particular, one goal of the book is to explain how MATLAB performs its sparse matrix computations.

Algorithms are presented in a mixture of pseudocode, MATLAB, and C, so knowledge of these is assumed. Also required is a basic knowledge of linear algebra, graph theory, algorithms, and data structures. A short review of these topics is provided. Each chapter includes a set of exercises to reinforce the topic.[3]

CSparse is written in C, using a spartan coding style. Using C instead of (say) Java or C++ allows for concise exposition, full disclosure of time and memory complexity, efficiency, and portability. CSparse can be downloaded from SIAM at www.siam.org/books/fa02. MATLAB 7.2 (R2006a) was used for this book. CSparse handles only real matrices and `int` integers. CXSparse is an extended version that includes support for real and complex matrices and `int` and `long` integers and can also be downloaded from www.siam.org/books/fa02.

The genesis of this book was a collection of lecture notes for a course on sparse matrix algorithms I taught at Stanford in 2003. I would like to thank Gene Golub, Esmond Ng, and Horst Simon for enabling me to spend a sabbatical at Stanford and Lawrence Berkeley National Laboratory for the 2002–2003 academic year. Several extended visits to Sandia National Laboratory at Mike Heroux's invitation enabled me to develop my versions of the left-looking sparse LU factorization algorithm and the Dulmage–Mendelsohn decomposition for use in Sandia's circuit simulation efforts. The algorithms presented here were developed with support from various sources, including Sandia National Laboratory, the National Science Foundation (ASC-9111263, DMS-9223088, DMS-9504974, DMS-9803599, and CCR-0203720), The MathWorks, Inc., and the University of Florida. I would like to thank David Bateman for adding support for complex matrices and `long` integers to CXSparse.

Nick Higham, Cleve Moler, and the other members of the Editorial Board of the SIAM Fundamentals of Algorithms book series encouraged me to turn these lecture notes and codes into the printed page before you by inviting me to write this book for the series. Finally, I would like to thank David Day, John Gilbert, Chen Greif, Nick Higham, Sara Murphy, Pat Quillen, David Riegelhaupt, Ken Stanley, Linda Thiel, and my Spring 2006 sparse matrix class (Suranjit Adhikari, Pawan Aurora, Okiemute Brume, Yanqing "Morris" Chen, Eric Dattoli, Bing Jian, Nick Lord, Siva Rajamanickam, and Ozlem Subakan), who provided helpful feedback on the content and presentation of the book.

Tim Davis
University of Florida, Gainesville, Florida
www.cise.ufl.edu/∼davis
April 2006

[3]Instructors: please do not post solutions on the web where they are publicly readable. Use a password-protected web page instead.